ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

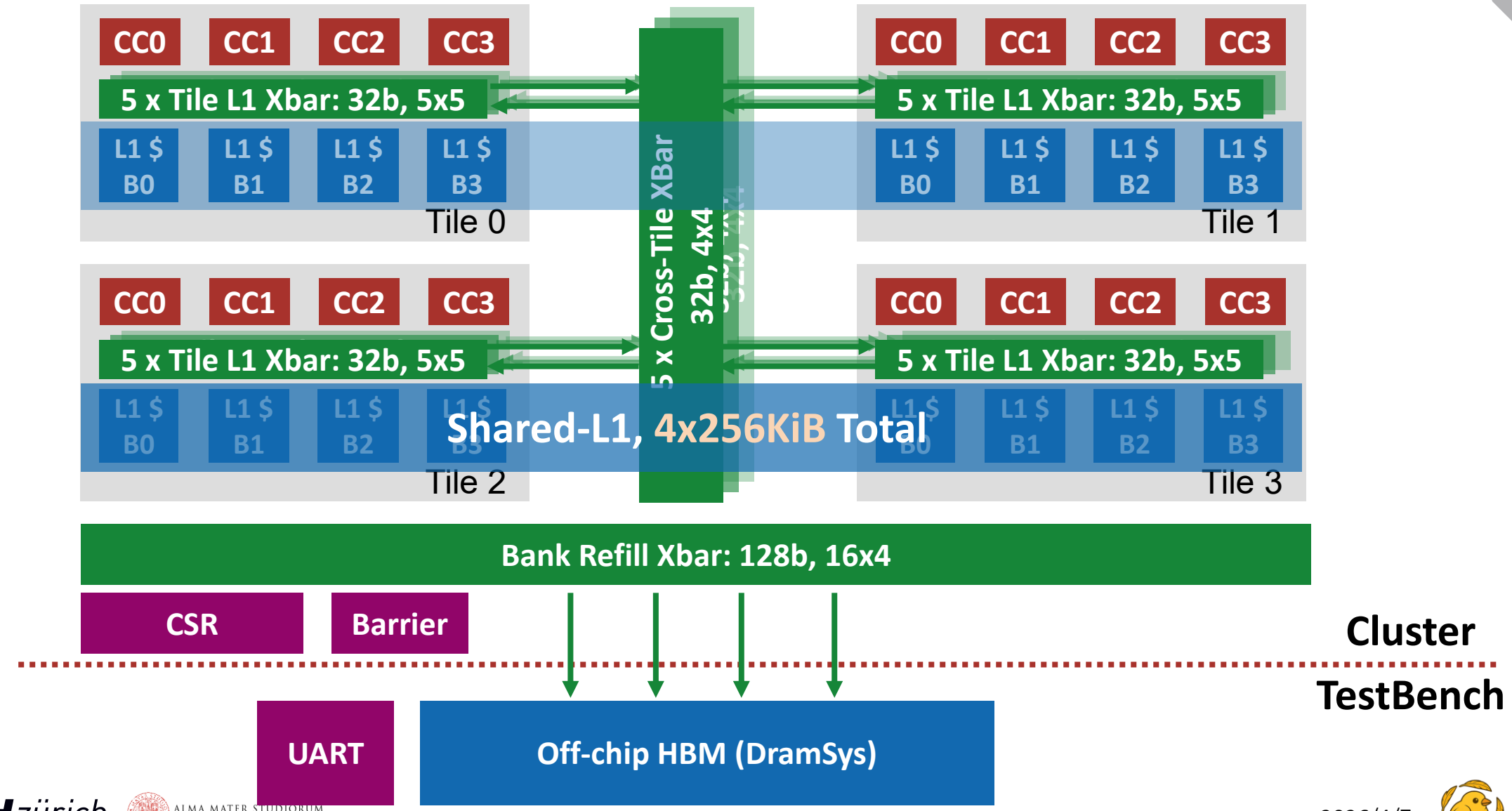
Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch

PULP Platform
Open Source Hardware, the way it should be!

@pulp_platform
pulp-platform.org
youtube.com/pulp_platform

Hardware Development (Cluster)



Hardware Development (Cluster)

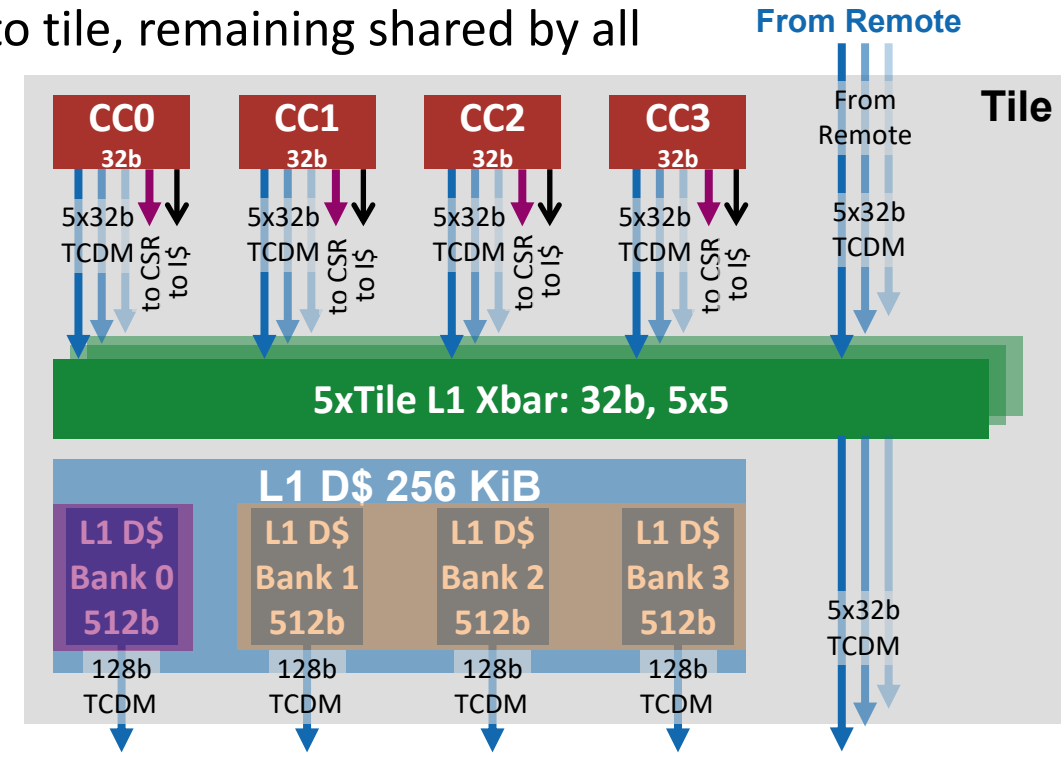
- **Cache Partition**

  Supported Patterns:

  - All Shared: shared by all tiles
  - All Private: private to local tile
  - Mixed: Specified number of banks local to tile, remaining shared by all

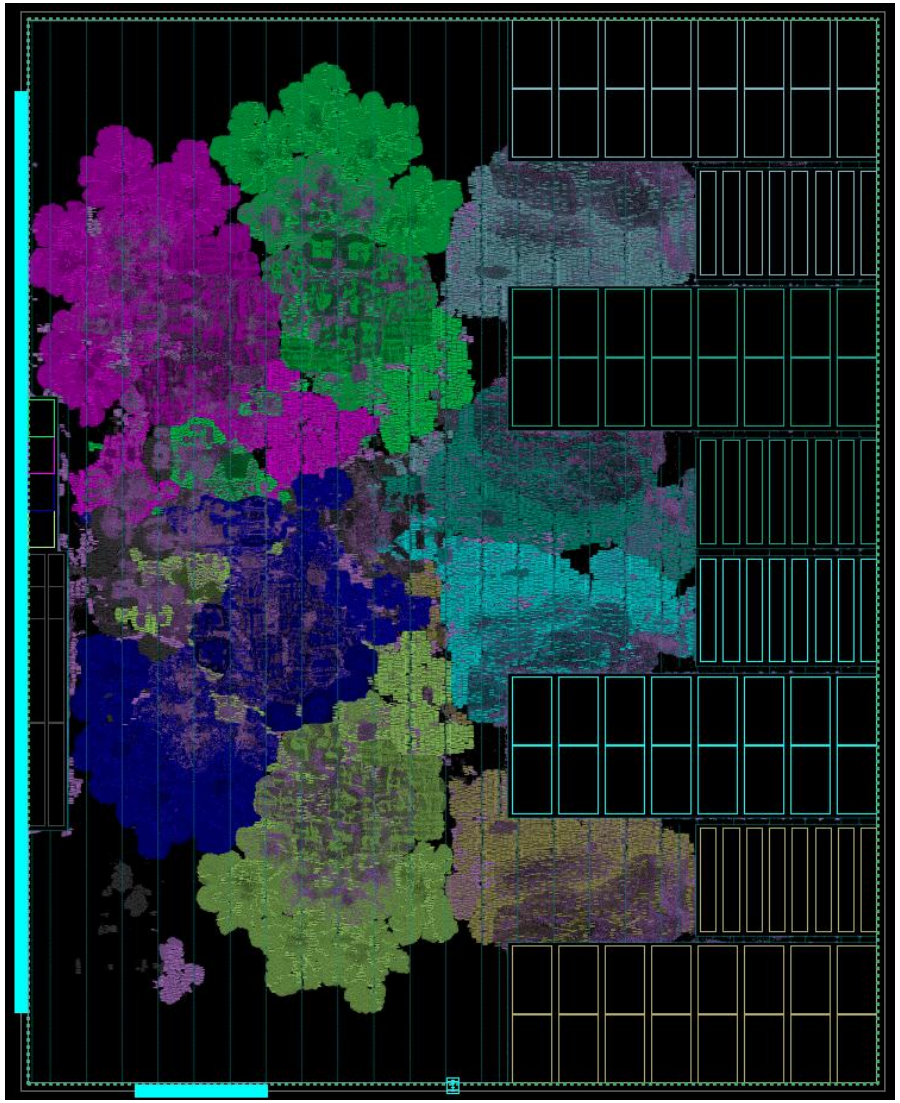
- **Cache Flush**
  - Flush/Init all
  - Flush shared
  - Flush private
    - Can config which tile(s) to flush

- **Have runtime integrated**



Hardware Development (Cluster)

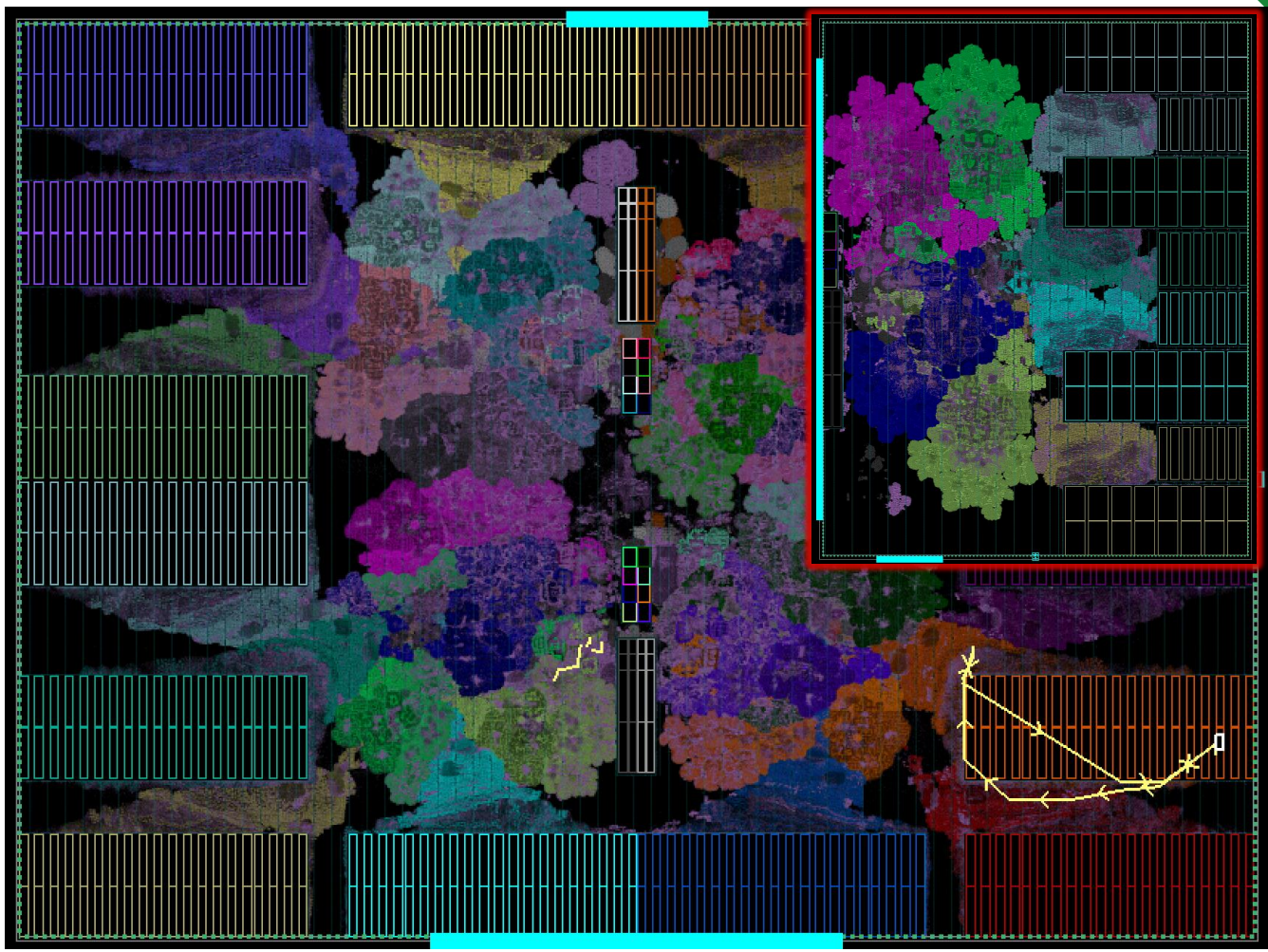
- **Folded Cache Backend**
  - Single tile now
  - ~50% area utilization
  - 5% logic area reduction
    Increased tag size due to byte-write support
  - Detailed area breakdown ongoing
  - Timing closed at TT @1GHz in 12nm
- **7nm Flow Ongoing (routing now)**



Hardware Development (Cluster)

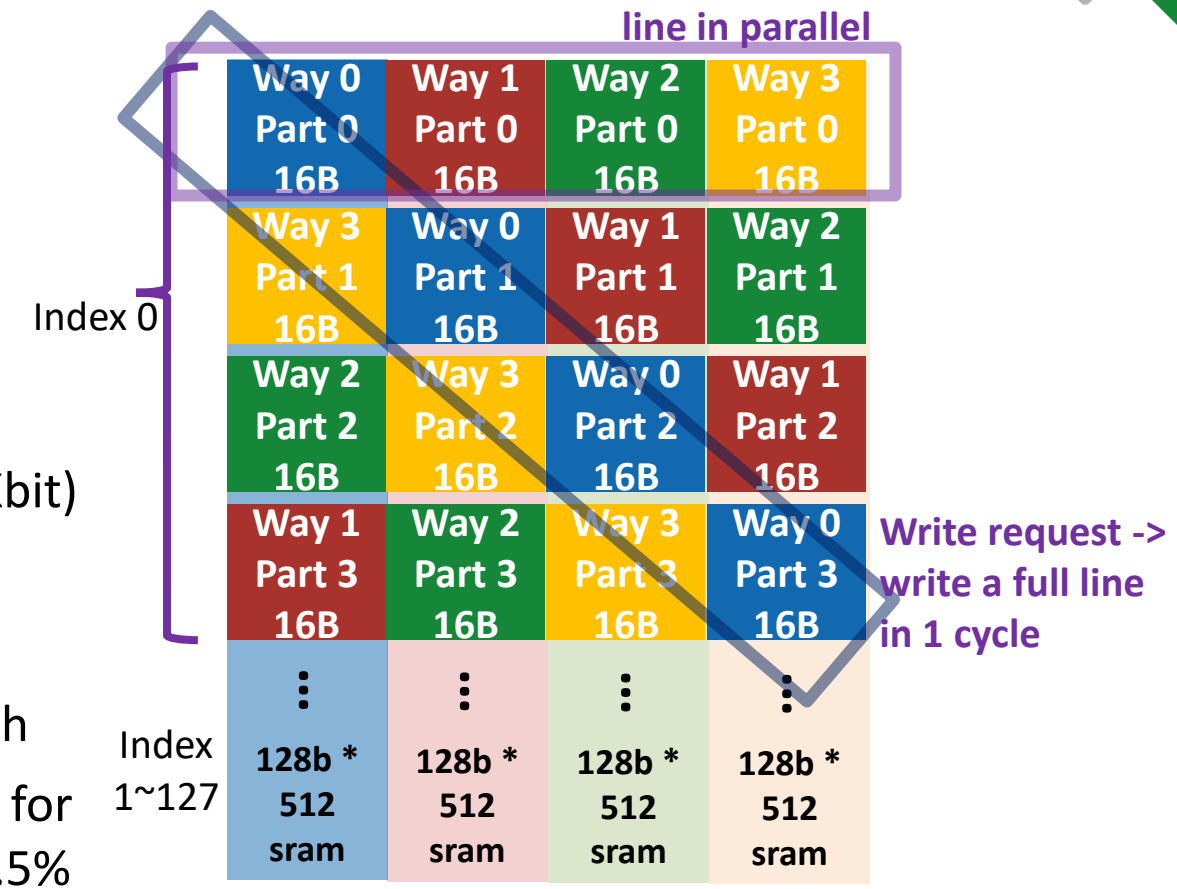
Same Scale: ~20%

- **Folded Cache Backend**
  - Single tile now
  - ~50% area utilization
  - 5% logic area reduction
    Increased tag size due to byte-write support
  - Detailed area breakdown ongoing
  - Timing closed at TT @1GHz in 12nm
- **7nm Flow Ongoing (routing now)**



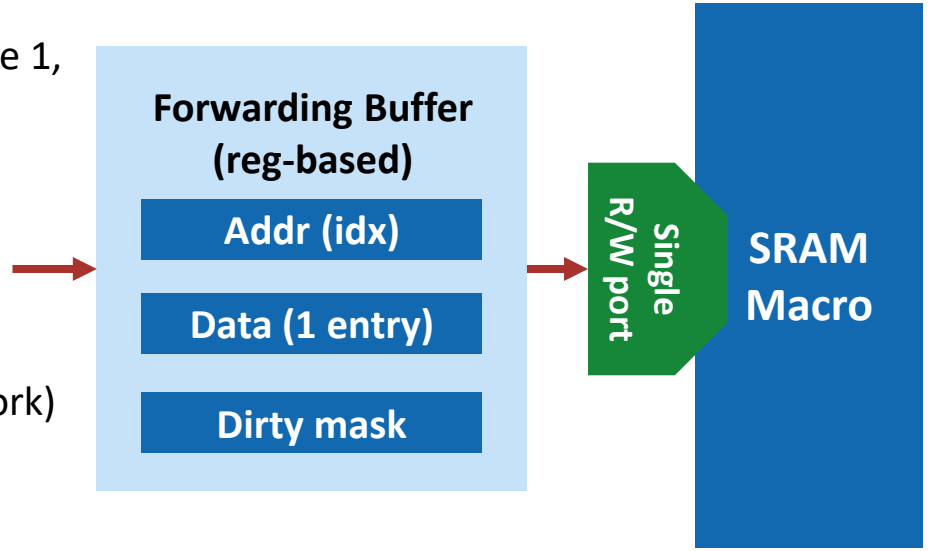
Hardware Development (L1 Cache)

- **Bandwidth Improvement Progress**
  - Replaced **LRU** replacement with the **hash-based pseudo-random** way-selection scheme
    - Direct way selection **removes LRU** metadata SRAM lookup/update overhead
    - Reads only one way instead of all four ways
  - Put dirty bits in reg rather than meta SRAM (~1 Kbit)
    - **Remove dirty** bit metadate SRAM lookup/update
  - Now the **long vector load** can have full bandwidth
  - Random replacement seems to be more suitable for RLC workloads, 100 RLC pkg (52551 -> 37048, 29.5% improvement)



Hardware Development (L1 Cache)

- **Bandwidth Improvement Ongoing**
  - The long vector store bandwidth is still a problem
    - For now, only half BW
    - The insitu cache controller assume the SRAM Macros don't have bit/byte enable signal
      - For a write a request, it read the full cache line from data SRAM at cycle 1, and write an updated cache line back to data SRAM at cycle 1
      - This data SRAM read then write cause a conflict with the next request
    - Forwarding Buffer can help
      - For hit, read/write concurrently
      - For miss, need handshake to stall the following request (need some work)
      - Currently, it only helps to reduce some SRAM access (better energy), haven't fully utilize its higher bandwidth.



RLC kernel PR Review

- **New RLC data structures**
  - RcvPktHeader (64B), DlschInd (32B), TestDataStru, UeStateRpt (2KB)
  - new fields in rlc_context_t: pingFlag, sduNum, sduBytes, tbsize, rlcthrp, dlPduNum, rlcOm[16], dlDelayInfo[16], …
- **New workload simulation**
  - ue_status_rpt(): Simulates receiving ACK from UE, processes sent_list every 2 packets
  - Extra cache-line reads/writes per packet: reads RcvPktHeader from node data, writes 64B back via vector_memcpy32_m4_opt, writes to rlcOm[16] and dlDelayInfo[16] arrays
  - Multiple new atomic_fetch_add operations per packet to simulate shared state updates (this can be replace by lock)
- **Progress**
  - Fixed syntax errors, now the kernel can be compiled
  - The kernel still does not run correctly, suspicion: new control-logic changes introduced a data race, debug in ongoing.

Next Step

- **InSitu-Cache Paper**
  - Both Zexin and Diyou will focus on the InSitu cache paper in March. The progress on other side might be delayed
- **Diyou Side**
  - 7nm Flow (ongoing)
  - Merge folded cache, collect benchmark and backend results
  - Started RTL development on multi-group
    Based on TeraNoC structure to integrate router
- **Zexin Side**
  - Fix the bandwidth issue introduced by folded cache design
  - Start the backend evaluation of the new cache controller design

Timeline

		2025						2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11
System	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] To be started soon													
L1 Access Latency	Hardware optimization (Cache & Interco) [RTL] Close to Complete													
	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL]													
Large NoC latency	Reduce per-hop latency [Physical] Ongoing by colleague													
	Cut clk-domain (optional) [RTL + Physical]													
	NoC Topologies [RTL + GVSoC + Physical]													

Timeline

		2025						2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11
Improve scalar core performance	Shared vector core [RTL + GVSoC]													
SW Kernel	Kernel optimization [SW] Delayed by one month, ongoing													
	Kernel development [SW]													
Report	Final Report and Benchmark													

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
    Currently has reduced to 6 cycles:
    Core => Xbar => AMO => Ctrl (2) => AMO => Core
  - **(Complete, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Ongoing, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Ongoing master thesis, now is working on the coherence between L1 caches
    => Currently plan to use hierarchical directory-based approach

L1 Access Latency

- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**
    Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
    => Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Planned, GVSoC & Physical) NoC Topology**
    Explore different NoC topologies
    => Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2

PE-Related

- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Planned, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.
    Discussed Solution: critical section

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**
    Implement the missing use cases for multi-user

Thank you!